ASCON su FPGA: Progettazione e ottimizzazione di un cifrario leggero per sistemi IoT sicuri ed efficienti

Dipartimento di Ingegneria informatica automatica e gestionale Antonio Ruberti

Sapienza Università di Roma

Riccardo D'Annibale
Matricola 2063306
Relatore
Prof. Riccardo Della Sala

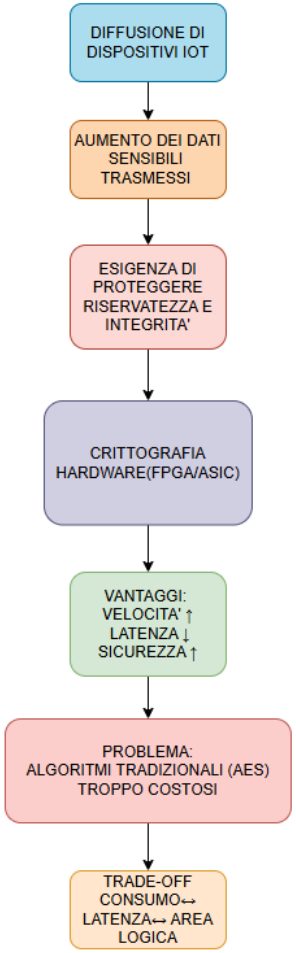
2024/2025

Outline

- Introduzione e contesto
- Obiettivo della tesi
- Ruolo dell’ ASCON
- Studio di fattibilità
- Implementazione su FPGA
- Risultati sperimentali
- Possibili sviluppi e conclusioni

Introduzione e contesto

Sicurezza hardware e vincoli nei dispositivi embedded



Introduzione e contesto

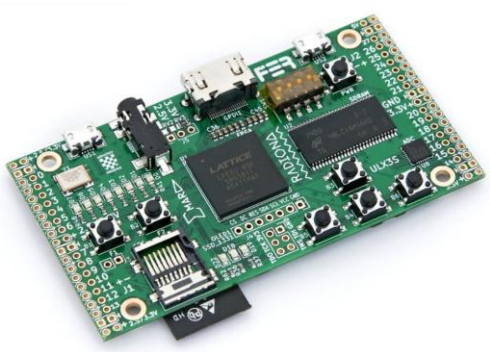
Crittografia leggera e contesti IoT

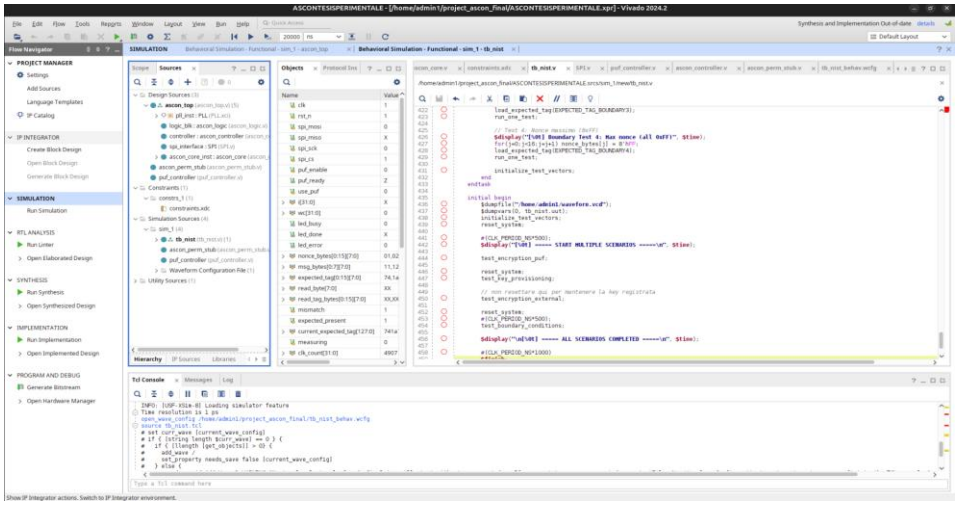
- Nasce così la **crittografia leggera**: il progetto si inserisce in questo ambito, con una focalizzazione sulle applicazioni IoT.
- È necessario trovare un compromesso tra sicurezza e complessità implementativa: soluzioni molto robuste ma pesanti.
- Le primitive *lightweight* mirano a ottenere il miglior rapporto possibile **area / throughput / power** .

Obiettivo della tesi

Progettare, implementare e **ottimizzare l' ASCON su FPGA**, utilizzando **Vivado** per sintesi e analisi post-implementazione.
Le ottimizzazioni mirano a:

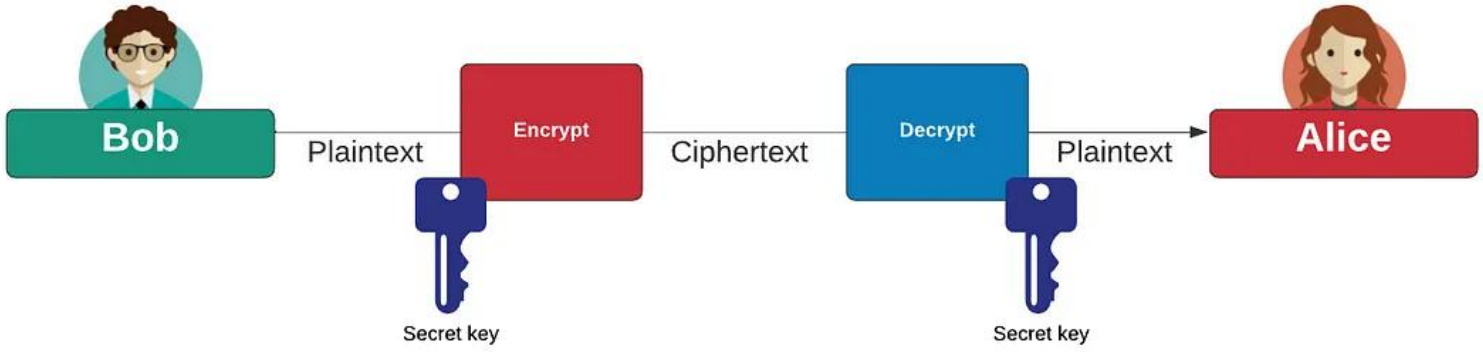
- ridurre area occupata e consumo energetico,
- aumentare throughput e mantenere bassa latenza,
- garantire conformità agli standard di sicurezza NIST.





Ruolo di ASCON

- ASCON è una famiglia di primitive **AEAD e hash** basata su una **costruzione sponge** e una **permutazione a 320 bit** (5×64 bit).
- Le istanze principali (**Ascon-128**, **Ascon-128a**) offrono **sicurezza a 128 bit** e sono pensate per implementazioni **compatte e low-power**.
- La stessa permutazione viene **riutilizzata** per AEAD, hash e XOF → logica più semplice e riuso hardware.
- Ideale per dispositivi **FPGA o IoT con risorse limitate**, dove risulta un'alternativa più leggera ad **AES-GCM** (schema AEAD classico).

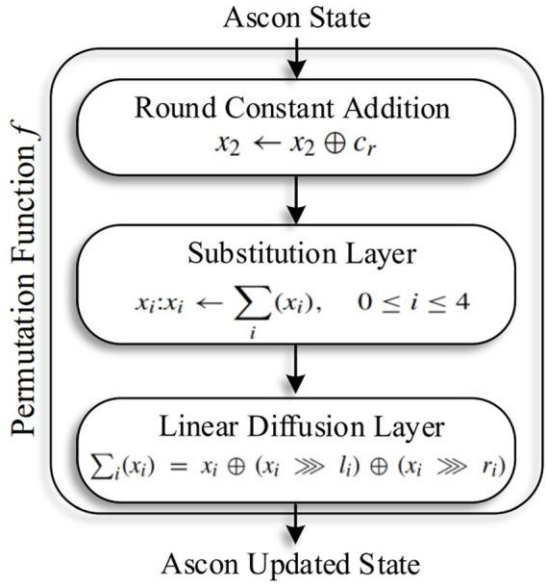


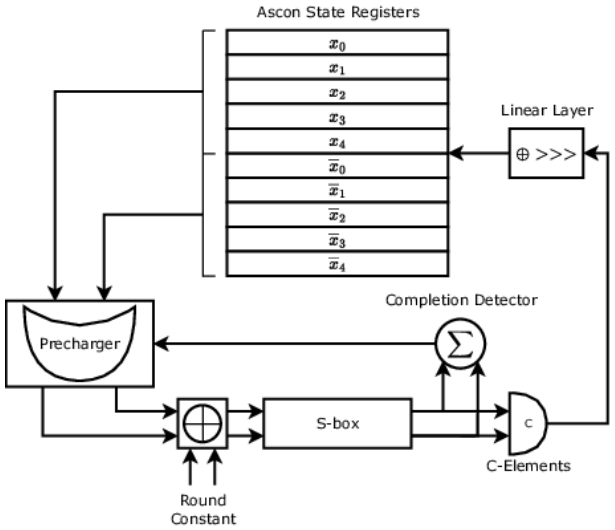
Struttura

Lo **stato** di ASCON è 320 bit = 5 parole da 64 bit: x0, x1, x2, x3, x4.

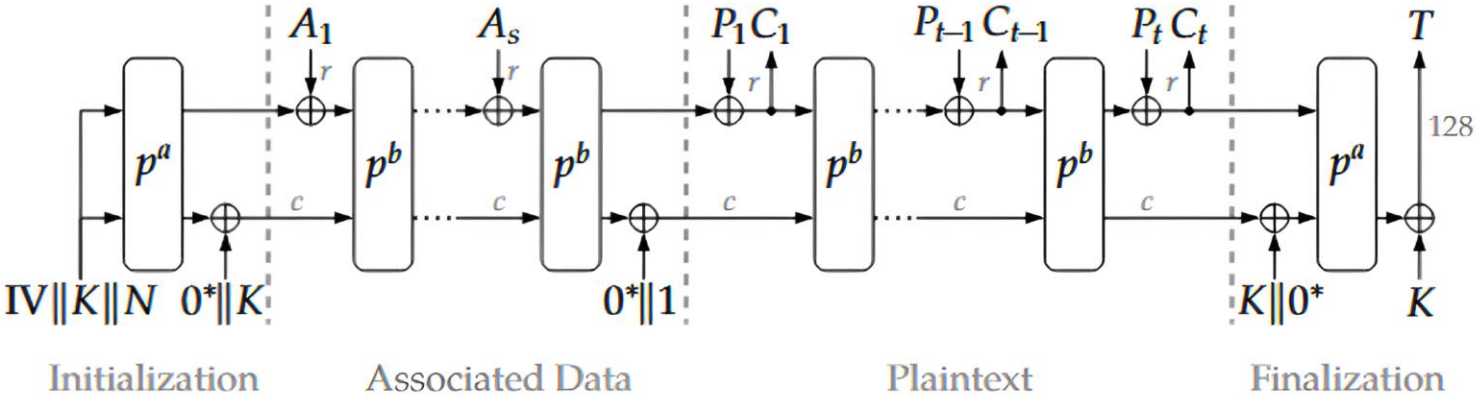
Ogni round applica tre sotto-passi (nell'ordine):

1. **Addizione del round constant** (si effettua l'operazione di XOR un byte costante su x2).
2. **Strato di sostituzione**: la S-box a 5 bit applicata in bit-slicing (cioè la stessa 5-bit S-box viene eseguita in parallelo su tutti i 64 bit delle parole).
3. **Strato di diffusione lineare**: per ogni parola si fa l'operazione di XOR con due rotazioni a destra di se stessa per diffondere i bit dentro la parola.

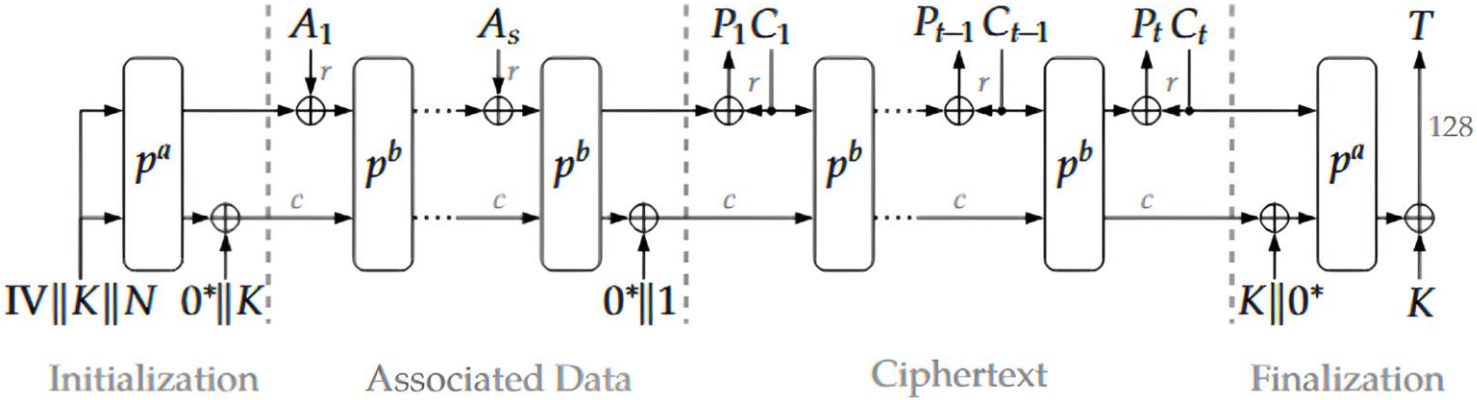




Encryption - Decryption

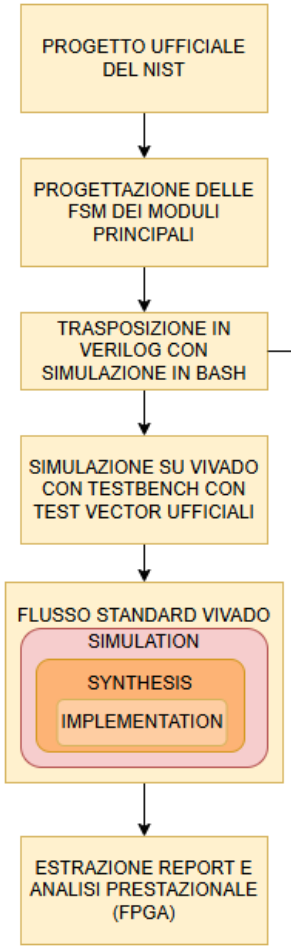


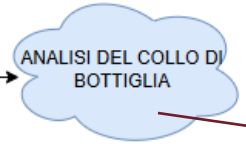
(a) Encryption $\mathcal{E}_{k,r,a,b}$

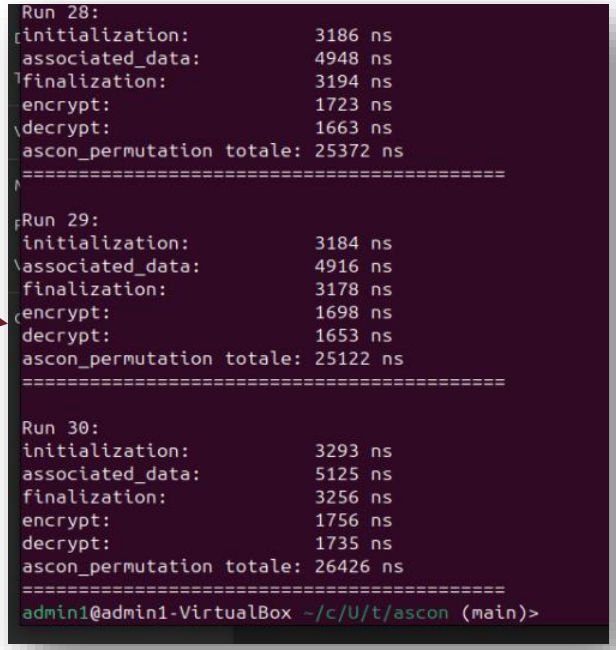


(b) Decryption $\mathcal{D}_{k,r,a,b}$

Implementazione su FPGA







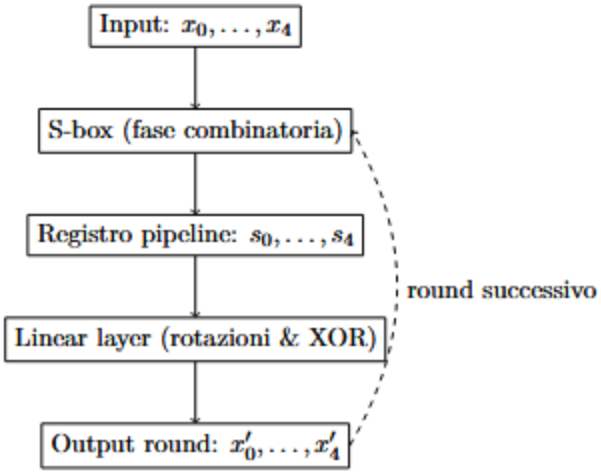
```
Run 28:
initialization:            3186 ns
associated_data:           4948 ns
finalization:              3194 ns
encrypt:                   1723 ns
decrypt:                   1663 ns
ascon_permutation totale: 25372 ns
==========================================

Run 29:
initialization:            3184 ns
associated_data:           4916 ns
finalization:              3178 ns
encrypt:                   1698 ns
decrypt:                   1653 ns
ascon_permutation totale: 25122 ns
==========================================

Run 30:
initialization:            3293 ns
associated_data:           5125 ns
finalization:              3256 ns
encrypt:                   1756 ns
decrypt:                   1735 ns
ascon_permutation totale: 26426 ns
==========================================
admin1@admin1-VirtualBox ~/c/U/t/ascon (main)>
```

Studio della fattibilità : OTTIMIZZAZIONE

1. **Ottimizzazione della permutazione**
   - Pipeline a 2 sottostadi per round: S-box (combinatoria) → registro intermedio s0..s4 → linear layer.
   - Interfaccia: start, rounds, busy, done; phase e round_ctr controllano i sottostadi.
   - Trade-off: +Fmax potenziale (meno logica combinatoria per ciclo) ma latency raddoppiata (12 round → 24 cicli).



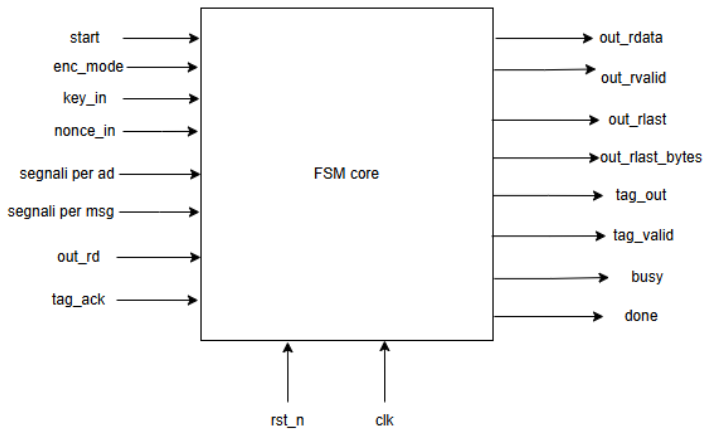
Studio della fattibilità : OTTIMIZZAZIONE

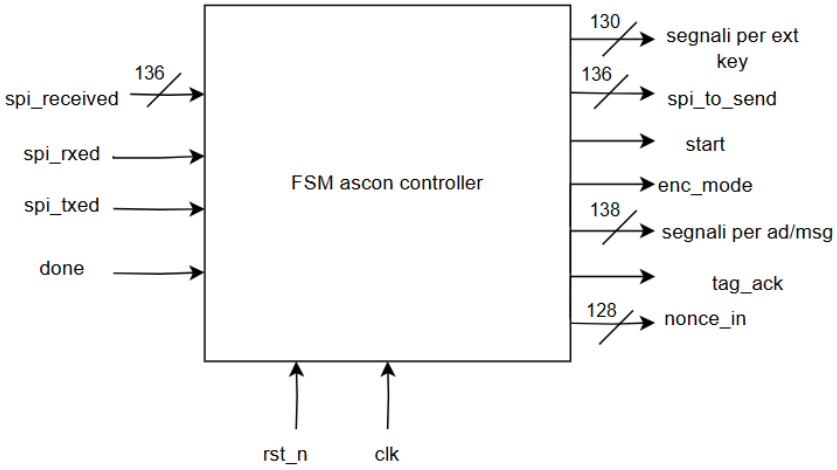
1. **Ottimizzazione della permutazione**
   - Pipeline a 2 sottostadi per round: S-box (combinatoria) → registro intermedio s0..s4 → linear layer.
   - Interfaccia: start, rounds, busy, done; phase e round_ctr controllano i sottostadi.
   - Trade-off: +Fmax potenziale (meno logica combinatoria per ciclo) ma latency raddoppiata (12 round → 24 cicli).
2. **Micro-ottimizzazioni e riuso**
   - ror64 centralizzata per rotazioni 64-bit.
   - Factoring logico nella S-box (segnali intermedi).
   - Padding corretto (little-endian) via pad_block.

Studio della fattibilità : OTTIMIZZAZIONE

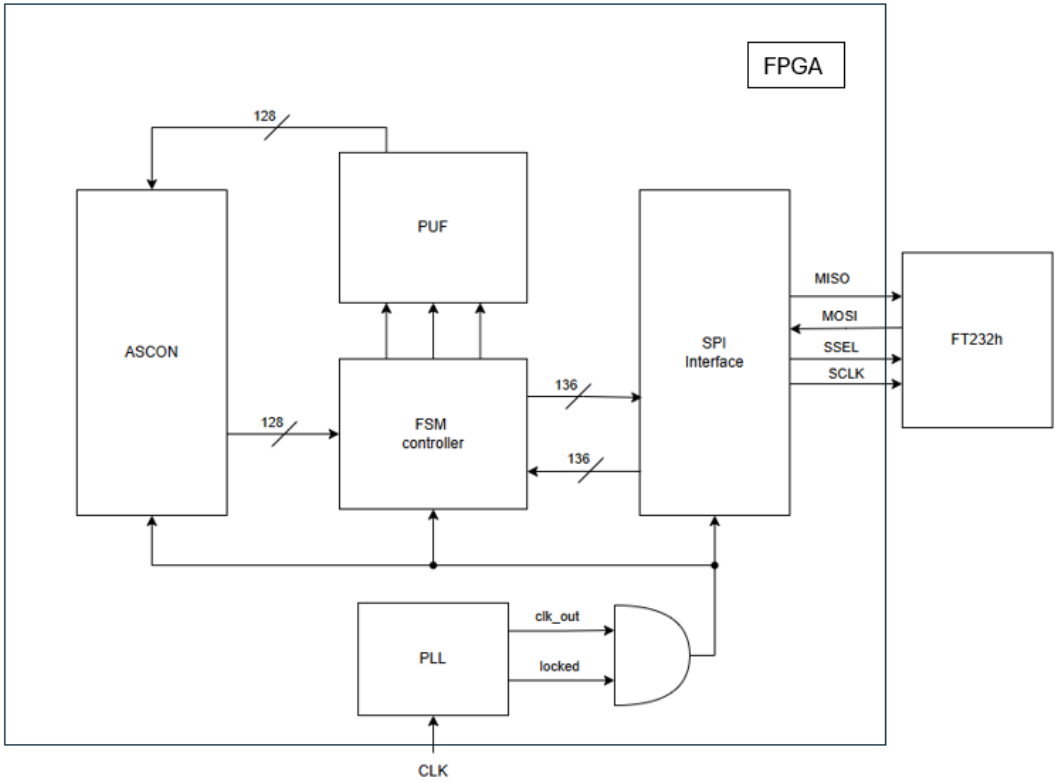
1. **Ottimizzazione della permutazione**
   - Pipeline a 2 sottostadi per round: S-box (combinatoria) → registro intermedio s0..s4 → linear layer.
   - Interfaccia: start, rounds, busy, done; phase e round_ctr controllano i sottostadi.
   - Trade-off: +Fmax potenziale (meno logica combinatoria per ciclo) ma latency raddoppiata (12 round → 24 cicli).
2. **Micro-ottimizzazioni e riuso**
   - ror64 centralizzata per rotazioni 64-bit.
   - Factoring logico nella S-box (segnali intermedi).
   - Padding corretto (little-endian) via pad_block.
3. **Buffering e interfacce**
   - FIFO locali implementate come array di reg (DEPTH = 16).
   - SPI robusto con sincronizzatori a 2 FF e rilevamento fronti (PE_SCK/NE_SCK).
   - PUF emulato senza hardware reale

Core ASCON e controller : FASE DI PROGETTO

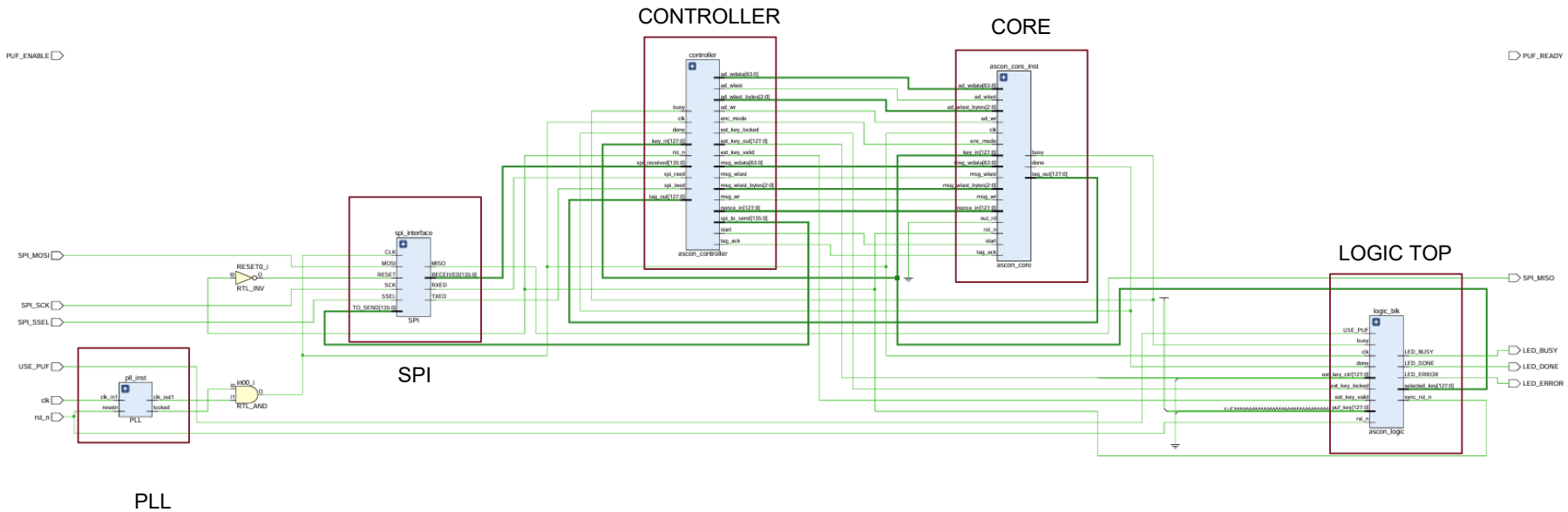




Architettura del design : FASE DI PROGETTO



Schematico Vivado



Waveform e Validazione dei Risultati di Simulazione

Encryption Test

Waveform e Validazione dei Risultati di Simulazione

Esempio di output ottenuto durante l’**encryption** e in un **test ai limiti (boundary test)**.

```
===== START MULTIPLE SCENARIOS =====
[5200000] --- Test Encryption with PUF ---
[5200000] ===== TEST STARTED (use_puf=1) =====
[19175000] Controller: SPI RXED byte = 0x81 (done_seen=0, done=0)
[19175000] Controller: Received CMD 0x81 (start op)
[74775000] Controller: Message length =     8 bytes
[297175000] Controller: Nonce loaded, starting processing
[311170000] ASCON Core: Tag produced = f861b9ce7dafb26ba834884365737a32
[408375000] Controller: All message data sent
[408380000] Controller: Processing done
[422375000] Controller: SPI RXED byte = 0x40 (done_seen=1, done=1)
[422375000] Controller: RD TAG cmd received, done_seen=1, done=1
[422375000] Controller: Preparing tag frame to send, tag_out=f861b9ce7dafb26ba834884365737a32
[422600000] Reading tag from DUT...
[436370000] Controller: Tag frame transmitted, asserting tag_ack
[436375000] Controller: SPI RXED byte = 0x00 (done_seen=0, done=1)
[436600000] Tag byte 0 = 0xf8, expected 0xf8
[436600000] Tag byte 1 = 0x61, expected 0x61
[436600000] Tag byte 2 = 0xb9, expected 0xb9
[436600000] Tag byte 3 = 0xce, expected 0xce
[436600000] Tag byte 4 = 0x7d, expected 0x7d
[436600000] Tag byte 5 = 0xaf, expected 0xaf
[436600000] Tag byte 6 = 0xb2, expected 0xb2
[436600000] Tag byte 7 = 0x6b, expected 0x6b
[436600000] Tag byte 8 = 0xa8, expected 0xa8
[436600000] Tag byte 9 = 0x34, expected 0x34
[436600000] Tag byte 10 = 0x88, expected 0x88
[436600000] Tag byte 11 = 0x43, expected 0x43
[436600000] Tag byte 12 = 0x65, expected 0x65
[436600000] Tag byte 13 = 0x73, expected 0x73
[436600000] Tag byte 14 = 0x7a, expected 0x7a
[436600000] Tag byte 15 = 0x32, expected 0x32
[436600000](*@\textcolor{red}{SUMMARY: Tag OK (all bytes match).}@*)
[436600000] ===== TEST COMPLETED =====
```

Bytes match

```
[1112600000] --- Boundary Conditions Tests ---
[1112600000] Boundary Test 1: Empty message (all zeros)
[1112600000] ===== TEST STARTED (use_puf=0) =====
[1126275000] Controller: SPI RXED byte = 0x81 (done_seen=0, done=0)
[1126275000] Controller: Received CMD 0x81 (start op)
[1181875000] Controller: Message length =     8 bytes
[1404275000] Controller: Nonce loaded, starting processing
[1418270000] ASCON Core: Tag produced = bc2c86545437ddd00ceeeb13b931d925
[1515475000] Controller: All message data sent
[1515480000] Controller: Processing done
[1529475000] Controller: SPI RXED byte = 0x40 (done_seen=1, done=1)
[1529475000] Controller: RD TAG cmd received, done_seen=1, done=1
[1529475000] Controller: Preparing tag frame to send, tag_out=bc2c86545437ddd00ceeeb13b931d925
[1529700000] Reading tag from DUT...
[1543470000] Controller: Tag frame transmitted, asserting tag_ack
[1543475000] Controller: SPI RXED byte = 0x00 (done_seen=0, done=1)
[1543700000] Tag byte 0 = 0xbc, expected 0xbc
[1543700000] Tag byte 1 = 0x2c, expected 0x2c
[1543700000] Tag byte 2 = 0x86, expected 0x86
[1543700000] Tag byte 3 = 0x54, expected 0x54
[1543700000] Tag byte 4 = 0x54, expected 0x54
[1543700000] Tag byte 5 = 0x37, expected 0x37
[1543700000] Tag byte 6 = 0xdd, expected 0xdd
[1543700000] Tag byte 7 = 0xd0, expected 0xd0
[1543700000] Tag byte 8 = 0x0c, expected 0x0c
[1543700000] Tag byte 9 = 0xee, expected 0xee
[1543700000] Tag byte 10 = 0xeb, expected 0xeb
[1543700000] Tag byte 11 = 0x13, expected 0x13
[1543700000] Tag byte 12 = 0xb9, expected 0xb9
[1543700000] Tag byte 13 = 0x31, expected 0x31
[1543700000] Tag byte 14 = 0xd9, expected 0xd9
[1543700000] Tag byte 15 = 0x25, expected 0x25
[1543700000] (*@\textcolor{red}{SUMMARY: Tag OK (all bytes match).}@*)
```

Bytes match

Timing Summary

Setup:

Metrica	Valore
Worst Hold Slack (WHS)	3.419 ns
Total Hold Slack (THS)	0 ns
Numero di failing endpoints	0
Numero totale di endpoints	10743

Hold:

Metrica	Valore
Worst Hold Slack (WHS)	0.314 ns
Total Hold Slack (THS)	0.000 ns
Numero di failing endpoints	0
Numero totale di endpoints	10743

Pulse-width:

Metrica	Valore
Worst Pulse Width Slack	4.5 ns
Total Pulse Width Negative Slack	0.000 ns
Numero di failing endpoints	0
Numero totale di endpoints	3642

Utilizzo risorse

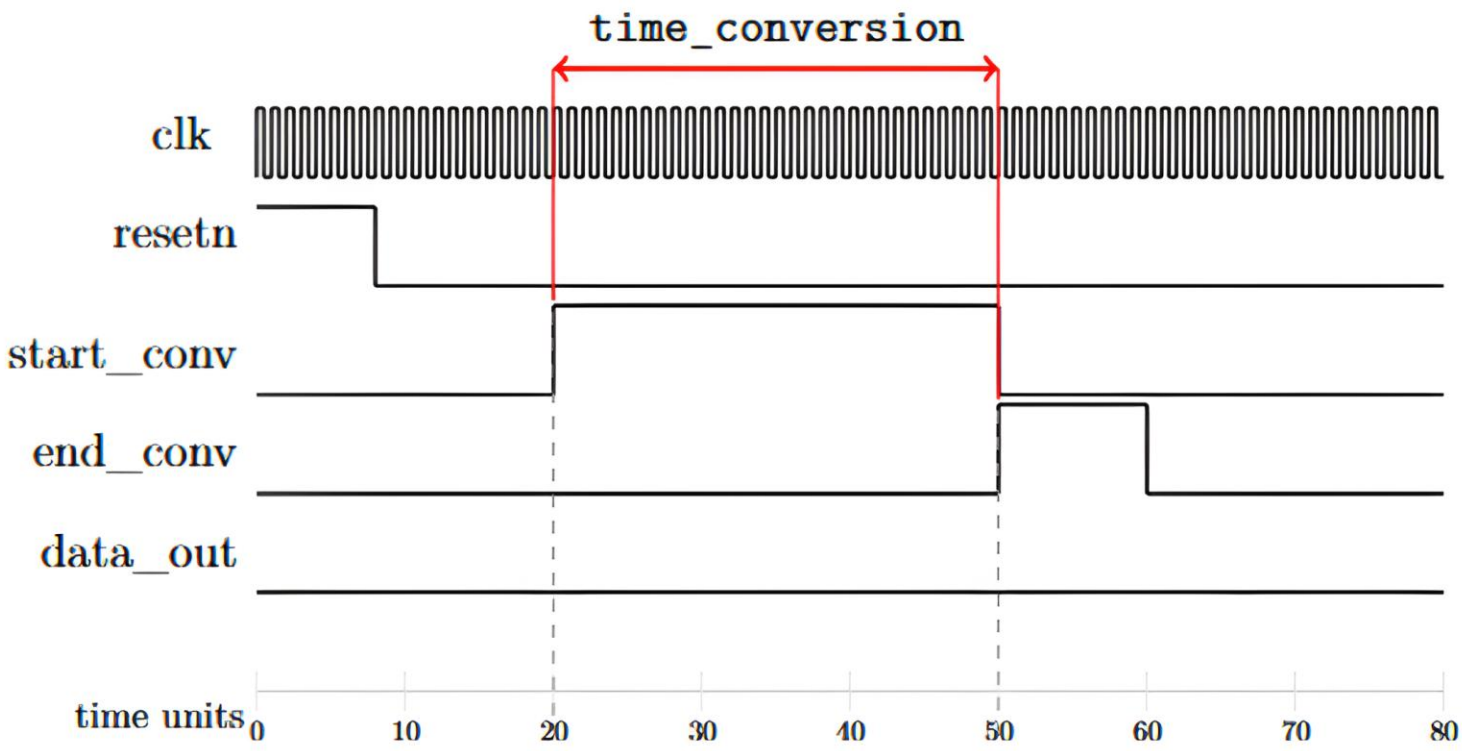
Risorsa	Utilizzo	Disponibili	Util (%)
LUT	1907	63400	3.01%
FF	3636	126800	2.87%
IO (Bonded IOB)	11	300	3.67%
BUFG	2	32	6.25%
MMCM / PLL	1	6	16.67%

Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs	RAMB36	RAMB18	DSP Blocks
ascon_top	(top)	1907	1907	0	0	3636	0	0	0
(ascon_top)	(top)	1	1	0	0	1	0	0	0
ascon_core_inst	ascon_core	1457	1457	0	0	2741	0	0	0
(ascon_core_inst)	ascon_core	403	403	0	0	1773	0	0	0
perm_inst	ascon_perm	1055	1055	0	0	968	0	0	0
controller	ascon_controller	360	360	0	0	598	0	0	0
logic_blk	ascon_logic	3	3	0	0	5	0	0	0
pll_inst	PLL	0	0	0	0	0	0	0	0
inst	PLL_clk_wiz	0	0	0	0	0	0	0	0
spi_interface	SPI	88	88	0	0	291	0	0	0

* Note: The sum of lower-level cells may be larger than their parent cells total, due to cross-hierarchy LUT combining

Calcolo throughput

$$\text{Throughput [bit/s]} = \frac{\text{Numero bit processati}}{\text{Tempo in secondi}}$$



Calcolo throughput

Dove:

$$T_{\text{op}} = N_{\text{cicli}} \cdot T_{\text{clk}} = \frac{N_{\text{cicli}}}{f_{\text{clk}}}$$

$$\text{Throughput} = \frac{\text{Frequenza del clock} \times \text{Numero di bit per blocco}}{\text{Numero di cicli per blocco}}$$

$$\text{Throughput} = \frac{50\ \text{MHz} \times 64\ \text{bit}}{4907\ \text{cicli}} \approx 0,652\ \text{Mbit/s}$$

Confronto con la letteratura

Implementazione	FPGA Target	LUT	FF	BRAM	BUFG	MMCM/PLL
Questo lavoro (ASCON + SPI + PUF simulato)	Artix-7 (XC7A100T)	1907	3636	–	2	1
Kurian & Chen (2025) **kurian2025ascon**	Artix-7 (Arty A7-100T)	3490	4285	32	–	–
Chen *et al.* (2023) **chen2023ascon**	Artix-7 (Arty A7-100T)	2988	2044	0	–	–
Mirigaldi *et al.* (2024) **mirigaldi2024chips**	Artix-7	1277	366	–	–	–

L'implementazione proposta utilizza una quantità di risorse pari o inferiore a quella delle soluzioni presenti in letteratura, nonostante includa moduli aggiuntivi come l'interfaccia SPI e la logica PUF emulata. Ciò dimostra l'efficienza della soluzione proposta e la sua compatibilità con piattaforme embedded leggere.

Analisi dei colli di bottiglia

- **SPI (host → core)** : Trasferimento byte-per-byte con overhead; diventa collo di bottiglia con molte transazioni brevi.
- **Controller FSM** : Cicli di parsing/assemblaggio non produttivi che aumentano la latenza.
- **Permutazione pipelined (2 cicli/round)** : Riduce complessità combinatoria e aumenta fmax, ma raddoppia i cicli per permutazione (trade-off tempo ↔ area).
- **FIFO in registri (DEPTH=16)** : Rischio di backpressure sui burst e consumo significativo di LUT/FF; BRAM/DMA preferibili per buffer profondi.
- **Limitazioni funzionali** : PUF solo emulata; zeroization parziale; assenti contromisure SCA.

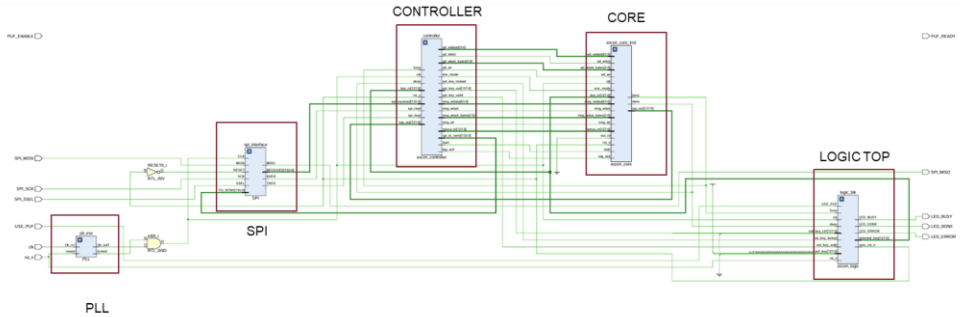
Possibili sviluppi futuri

- **Approfondimento sulla robustezza del design rispetto a PVT e attacchi SC**: temperatura e contromisure SCA(DPA/SPA).
- **Zeroization completa**: azzerare completamente x0…x4,chiavi,nonce e buffer sensibili.
- **Incremento del throughput**: deep pipeline di alcune operazioni del controller.

Conclusioni

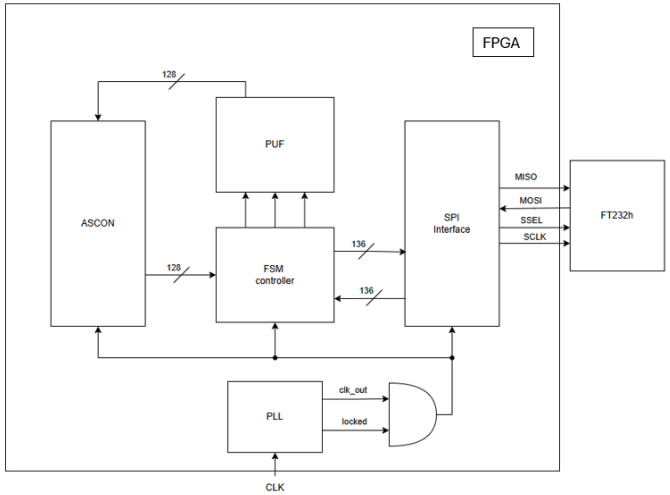
- Realizzata **implementazione hardware completa e funzionante** di **ASCON-128** su FPGA **Artix-7**
- Architettura **modulare e verificabile**, integrata con **SPI, FIFO, PLL e PUF emulata**
- **Validazione** : comportamento corretto in tutti gli scenari di test
- Design **leggero** (1907 LUT,3636 FF) e ben **ottimizzato**.
- Facile estensione per ottimizzazioni e nuove features.

Grazie a tutti per l’attenzione



Instance	Module	Total LUTs	Logic LUTs	LUTRAMs	SRLs	FFs	RAMB36	RAMB18	DSP Blocks
ascon_top	(top)	1907	1907	0	0	3636	0	0	0
(ascon_top)	(top)	1	1	0	0	1	0	0	0
ascon_core_inst	ascon_core	1457	1457	0	0	2741	0	0	0
(ascon_core_inst)	ascon_core	403	403	0	0	1773	0	0	0
perm_inst	ascon_perm	1055	1055	0	0	968	0	0	0
controller	ascon_controller	360	360	0	0	598	0	0	0
logic_blk	ascon_logic	3	3	0	0	5	0	0	0
pll_inst	PLL	0	0	0	0	0	0	0	0
inst	PLL_clk_wiz	0	0	0	0	0	0	0	0
spi_interface	SPI	88	88	0	0	291	0	0	0

* Note: The sum of lower-level cells may be larger than their parent cells total, due to cross-hierarchy LUT combining



Metrica	Valore
Worst Hold Slack (WHS)	3.419 ns
Total Hold Slack (THS)	0 ns
Numero di failing endpoints	0
Numero totale di endpoints	10743

(Setup)

Metrica	Valore
Worst Hold Slack (WHS)	0.314 ns
Total Hold Slack (THS)	0.000 ns
Numero di failing endpoints	0
Numero totale di endpoints	10743

(Hold)

Metrica	Valore
Worst Pulse Width Slack	4.5 ns
Total Pulse Width Negative Slack	0.000 ns
Numero di failing endpoints	0
Numero totale di endpoints	3642

(Pulse-width)

Risorsa	Utilizzo	Disponibili	Util (%)
LUT	1907	63400	3.01%
FF	3636	126800	2.87%
IO (Bonded IOB)	11	300	3.67%
BUFG	2	32	6.25%
MMCM / PLL	1	6	16.67%

Riccardo D’Annibale : 2063306